Proyectos de Programación 26/27
Práctica de pruebas – 16.11.2026

En esta práctica vas a aprender a **escribir pruebas con GoogleTest** como las que pide la Entrega 2: qué es una prueba unitaria, de caja negra, de caja blanca, de integración y de rendimiento, cómo se escribe cada una y, sobre todo, **para qué sirve una prueba: para encontrar errores antes que el usuario**.

Trabajaréis **por parejas**, con dos repositorios públicos que ya están preparados (no tienes que traer nada):

Repositorio	Qué es	Qué haces con él
Referencia `pp-lab3-tests-referencia`	Un **hotel** (Habitacion, Suite, Hotel, guardar/cargar) con **24 pruebas** ya hechas y bien hechas	Lo descargas, lo compilas y lo **lees**. No se modifica: es el modelo del que copiar
Ejercicio `pp-lab3-tests-ejercicio`	Una **biblioteca** (Libro, Novedad, Biblioteca, guardar/cargar) con **la misma estructura**, pero con **un solo test**	Creáis vuestra copia y escribís las pruebas **copiando y adaptando** las del hotel

> **Atención**
>
> **El código de la biblioteca tiene 4 errores escondidos.** No los busquéis leyendo el código: si vuestras pruebas están bien hechas, **ellas los encontrarán** (se pondrán en rojo). La regla de la práctica es: **primero el test en rojo; después, el arreglo**.

> **Cómo leer esta guía**
>
> - Los recuadros negros son la **terminal**. Lo que tienes que escribir está en `amarillo`; lo que va delante (`PS C:\Users\alumno>` o `alumno@MacBook … %`) es el *prompt*: **no lo escribas**. Lo que aparece en gris debajo es lo que responde el ordenador. Los `…` son líneas que se han omitido.
> - Cuando Windows y Mac son distintos verás dos recuadros: **usa solo el de tu sistema**.
> - Los recuadros verdes dicen lo que deberías ver; los rojos, qué hacer si ves otra cosa.
> - **Por parejas:** trabajad en **un solo ordenador** y **cambiad de quién escribe en cada paso**. El otro lee la guía y revisa.

Parte	Qué haces	Tiempo
1	Comprobar las herramientas y descargar la referencia	10 min
2	Recorrer las pruebas de la referencia y ver una en rojo a propósito	10 min
3	Crear vuestro repositorio de ejercicio	10 min
4-7	Escribir las pruebas de la biblioteca y encontrar los 4 errores	60-70 min
8	(Opcional) Prueba de rendimiento	—
Pistas	Si lleváis más de 10 minutos atascados con un error: qué está mal, por qué y cómo se arregla	—
Para casa	Lo mismo en el proyecto de vuestro grupo	—

Pirineos, 55 - 28040 Madrid - Tel. 91 444 51 61 www.cunef.edu

1. Preparar el equipo y descargar la referencia (10 min)

1.1 Comprobar que tienes las herramientas

Son las mismas que en la práctica de CMake. Cada comando debe responder con un número de versión, **no con un error**:

Windows · PowerShell

```
PS C:\Users\alumno\PP> g++ --version
g++.exe (Rev2, Built by MSYS2 project) 14.2.0
PS C:\Users\alumno\PP> cmake --version
cmake version 3.31.0
PS C:\Users\alumno\PP> git --version
git version 2.51.0.windows.2
PS C:\Users\alumno\PP> ninja --version
1.12.1
```

Mac · Terminal

```
alumno@MacBook-de-alumno PP % g++ --version
Apple clang version 17.0.0 (clang-1700.0.13.3)
alumno@MacBook-de-alumno PP % cmake --version
cmake version 4.1.2
alumno@MacBook-de-alumno PP % git --version
git version 2.39.5 (Apple Git-154)
```

Si no tienes la carpeta `PP`, créala en tu carpeta personal con `mkdir PP` y entra con `cd PP`. Los números de versión pueden ser distintos. **En Mac no hace falta `ninja`.**

> **Atención**
>
> **¿Te falta alguna herramienta?** Tienes el **Anexo · Instalar las herramientas** al final de esta guía. Instalar lleva unos 15-20 minutos: mientras tanto, trabajad en el ordenador de tu pareja.
>
> **Windows, `ninja` no se reconoce:** en la terminal **MSYS2 MSYS**, `pacman -S --needed mingw-w64-x86_64-ninja`, y después cierra y abre PowerShell.

1.2 Descargar, compilar y probar la referencia

Desde tu carpeta `PP`:

Windows · PowerShell

```
PS C:\Users\alumno\PP> git clone https://github.com/jcor-cunpir/pp-lab3-tests-referencia.git
Cloning into 'pp-lab3-tests-referencia'...
…
PS C:\Users\alumno\PP> cd pp-lab3-tests-referencia
PS C:\Users\alumno\PP\pp-lab3-tests-referencia> cmake -S . -B build -G Ninja
…
-- Build files have been written to: C:\Users\alumno\PP\pp-lab3-tests-referencia\build
PS C:\Users\alumno\PP\pp-lab3-tests-referencia> cmake --build build
…
[…] Linking CXX executable tests_gestor.exe
```

Mac · Terminal

```
alumno@MacBook-de-alumno PP % git clone https://github.com/jcor-cunpir/pp-lab3-tests-referencia.git
Cloning into 'pp-lab3-tests-referencia'...
…
alumno@MacBook-de-alumno PP % cd pp-lab3-tests-referencia
alumno@MacBook-de-alumno pp-lab3-tests-referencia % cmake -S . -B build
…
-- Build files have been written to: /Users/alumno/PP/pp-lab3-tests-referencia/build
alumno@MacBook-de-alumno pp-lab3-tests-referencia % cmake --build build
…
[100%] Built target tests_gestor
```

La primera vez, `cmake -S . -B build` **descarga GoogleTest** de internet: tarda un poco más. Ahora, los tests:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab3-tests-referencia> cd build
PS C:\Users\alumno\PP\pp-lab3-tests-referencia\build> ctest --output-on-failure
    Start 1: GestorTests
1/1 Test #1: GestorTests ......................   Passed    0.40 sec

100% tests passed, 0 tests failed out of 1
PS C:\Users\alumno\PP\pp-lab3-tests-referencia\build> cd ..
```

Mac · Terminal

```
alumno@MacBook-de-alumno pp-lab3-tests-referencia % cd build
alumno@MacBook-de-alumno build % ctest --output-on-failure
    Start 1: GestorTests
1/1 Test #1: GestorTests ......................   Passed    0.40 sec

100% tests passed, 0 tests failed out of 1
alumno@MacBook-de-alumno build % cd ..
```

> **¿Qué ha pasado?**
>
> `ctest` dice «1 test» porque para CMake hay **un solo** programa de pruebas, `tests_gestor`, que dentro tiene **todos** los tests de GoogleTest. Para verlos uno a uno, se ejecuta directamente:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab3-tests-referencia> .\build\tests_gestor.exe
[==========] Running 24 tests from 5 test suites.
…
[ RUN      ] SuiteTest.ConDescuentoJustoConSieteNoches
[       OK ] SuiteTest.ConDescuentoJustoConSieteNoches (0 ms)
…
[==========] 24 tests from 5 test suites ran. (12 ms total)
[  PASSED  ] 24 tests.
```

Mac · Terminal

```
alumno@MacBook-de-alumno pp-lab3-tests-referencia % ./build/tests_gestor
[==========] Running 24 tests from 5 test suites.
…
[ RUN      ] SuiteTest.ConDescuentoJustoConSieteNoches
[       OK ] SuiteTest.ConDescuentoJustoConSieteNoches (0 ms)
…
[==========] 24 tests from 5 test suites ran. (12 ms total)
[  PASSED  ] 24 tests.
```

✔ **Deberías ver**

`[ PASSED ] 24 tests.` al final. Abre la carpeta en VS Code con `code .` para la parte 2.

✘ **Si en vez de eso ves...**

`Failed to clone repository: 'https://github.com/google/googletest.git'` : CMake necesita **internet** y **Git** para descargar GoogleTest. Comprueba la conexión y `git --version` , borra la carpeta `build` y vuelve a configurar.

`does not appear to contain CMakeLists.txt` : no estás en la carpeta del repositorio. `pwd` y `cd` .

`Does not match the generator used previously` : borra la carpeta `build` y vuelve a configurar.

Pirineos, 55 - 28040 Madrid - Tel. 91 444 51 61 www.cunef.edu

2. Recorrer las pruebas de la referencia (10 min)

En VS Code, abre la carpeta `tests/`. Hay un fichero por clase, y cada uno es un **tipo de prueba** distinto (lo que se ha visto en clase):

Fichero	Tipo de prueba	La idea
`test_habitacion.cpp`	**Unitaria, caja negra**	Se prueba **una clase sola**, mirando solo lo que **promete** su `.hpp` (sin leer el `.cpp`): casos normales y **valores límite** (el 0, el primer valor válido, el primero no válido)
`test_suite.cpp`	**Unitaria, caja blanca**	Se escribe **mirando el código**: si hay un `if`, un test por cada camino, y otro **justo en el límite** del `if`
`test_hotel.cpp`	**Unitaria con *fixture***	`TEST_F`: todos los tests parten del **mismo hotel** preparado en `SetUp()`, sin repetirlo en cada test
`test_almacen.cpp`	**Integración**	Varias piezas **juntas**: se guarda un hotel en un fichero real, se carga y se comprueba que vuelve igual
`test_rendimiento.cpp`	**Rendimiento**	No mira *qué* devuelve, sino **cuánto tarda** con muchos datos

2.1 La anatomía de un test

`tests/test_habitacion.cpp`

```
TEST(HabitacionTest, ReservarDosVecesDevuelveFalse) {
    Habitacion h(101, 80.0);          // 1. PREPARAR: crear los objetos
    h.reservar();                     // 2. ACTUAR: llamar a lo que se prueba
    EXPECT_FALSE(h.reservar());       // 3. COMPROBAR: ¿pasa lo que promete el .hpp?
    EXPECT_TRUE(h.estaOcupada());
}
```

`TEST(Grupo, Nombre)`: el grupo suele ser la clase (`HabitacionTest`) y el nombre dice **qué comprueba**, como una frase. **No hay `main()`**: lo pone GoogleTest (`GTest::gtest_main` en el `CMakeLists.txt`).

Quiero comprobar…	Macro	Ejemplo en la referencia
Que dos valores son iguales (`int`, `string`, punteros…)	`EXPECT_EQ(real, esperado)`	`EXPECT_EQ(h.getNumero(), 101);`
Que dos `double` son iguales	`EXPECT_DOUBLE_EQ(real, esperado)`	`EXPECT_DOUBLE_EQ(s.precioEstancia(7), 1260.0);`
Que algo es verdadero / falso	`EXPECT_TRUE` / `EXPECT_FALSE`	`EXPECT_FALSE(h.estaOcupada());`
Que una llamada lanza una excepción de ese tipo	`EXPECT_THROW(llamada, tipo)`	`EXPECT_THROW(Habitacion(101, 0.0), std::invalid_argument);`
Que una llamada **no** lanza	`EXPECT_NO_THROW(llamada)`	`EXPECT_NO_THROW(Habitacion(101, 0.01));`
Lo mismo, pero **parando el test** si falla	`ASSERT_…`	`ASSERT_NE(h, nullptr);` antes de usar `h->`

> **¿Qué ha pasado?**
>
> **Excepciones** (se explican el miércoles 18): cuando el constructor recibe datos que no valen, en lugar de crear un objeto incorrecto **«protesta»** con `throw std::invalid_argument(...)`. `EXPECT_THROW` comprueba que protesta cuando debe.

2.2 Ver un test en rojo, a propósito

Un test sirve cuando **falla si el código está mal**. Vamos a verlo: en `tests/test_habitacion.cpp`, línea 56, cambia `240.0` por `250.0`, **guarda** (`Ctrl+S` / `⌘S`), compila y pasa los tests:

`tests/test_habitacion.cpp, línea 56`

```
    EXPECT_DOUBLE_EQ(h.precioEstancia(3), 250.0);      // antes: 240.0
```

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab3-tests-referencia> cmake --build build
PS C:\Users\alumno\PP\pp-lab3-tests-referencia> .\build\tests_gestor.exe
…
C:/Users/alumno/PP/pp-lab3-tests-referencia/tests/test_habitacion.cpp:56: Failure
Expected equality of these values:
  h.precioEstancia(3)
    Which is: 240
  250.0
    Which is: 250
[  FAILED  ] HabitacionTest.PrecioEstanciaMultiplicaPorLasNoches (0 ms)
…
 1 FAILED TEST
```

Mac · Terminal

```
alumno@MacBook-de-alumno pp-lab3-tests-referencia % cmake --build build
alumno@MacBook-de-alumno pp-lab3-tests-referencia % ./build/tests_gestor
…
/Users/alumno/PP/pp-lab3-tests-referencia/tests/test_habitacion.cpp:56: Failure
Expected equality of these values:
  h.precioEstancia(3)
    Which is: 240
  250.0
    Which is: 250
[  FAILED  ] HabitacionTest.PrecioEstanciaMultiplicaPorLasNoches (0 ms)
…
 1 FAILED TEST
```

¿Qué ha pasado?

Así se lee un rojo: **fichero y línea** del `EXPECT` que ha fallado (`:56`), **qué se ha obtenido** (`h.precioEstancia(3)` → *Which is: 240*) y **qué se esperaba** (`250.0`), y el **nombre del test**. Aquí el que está mal es el test (3 × 80 = 240). En la biblioteca será al revés: **el test estará bien y lo que estará mal será la clase**.

Atención

Deja la línea 56 como estaba (`240.0`), guarda y comprueba que vuelven a pasar los 24. La referencia no se modifica.

3. Crear vuestro repositorio de ejercicio (10 min)

3.1 Crearlo desde la plantilla (lo hace UNA persona de la pareja)

1. Entra en **https://github.com/jcor-cunpir/pp-lab3-tests-ejercicio** con tu cuenta de GitHub.
2. Botón verde **Use this template** → **Create a new repository**.
3. *Repository name*: `lab3-biblioteca`. Da igual **Public** o **Private**. Pulsa **Create repository**.
4. Opcional: **Settings** → **Collaborators** → **Add people** para añadir a tu pareja (así también puede subir cambios desde su ordenador).
5. Pestaña **Actions**: al cabo de uno o dos minutos, la primera ejecución de **CI** termina en ✔ **verde**.

3.2 Descargarlo, compilarlo y probarlo

Desde tu carpeta `PP` (en VS Code: **Terminal** → **Nueva terminal**, y `cd ..` para salir de la referencia). Cambia `tu-usuario` por tu usuario de GitHub:

Windows · PowerShell

```
PS C:\Users\alumno\PP> git clone https://github.com/tu-usuario/lab3-biblioteca.git
PS C:\Users\alumno\PP> cd lab3-biblioteca
PS C:\Users\alumno\PP\lab3-biblioteca> cmake -S . -B build -G Ninja
…
PS C:\Users\alumno\PP\lab3-biblioteca> cmake --build build
…
PS C:\Users\alumno\PP\lab3-biblioteca> .\build\tests_gestor.exe
…
[  PASSED  ] 1 test.
PS C:\Users\alumno\PP\lab3-biblioteca> code .
```

Mac · Terminal

```
alumno@MacBook-de-alumno PP % git clone https://github.com/tu-usuario/lab3-biblioteca.git
alumno@MacBook-de-alumno PP % cd lab3-biblioteca
alumno@MacBook-de-alumno lab3-biblioteca % cmake -S . -B build
…
alumno@MacBook-de-alumno lab3-biblioteca % cmake --build build
…
alumno@MacBook-de-alumno lab3-biblioteca % ./build/tests_gestor
…
[  PASSED  ] 1 test.
alumno@MacBook-de-alumno lab3-biblioteca % code .
```

Se abre **otra ventana** de VS Code con la biblioteca. Trabajaréis con las dos a la vez: la **referencia** para copiar y la **biblioteca** para escribir. A partir de aquí, usa la terminal de la ventana de la biblioteca.

3.3 Las equivalencias

Hotel (referencia)	Biblioteca (ejercicio)
`Habitacion(numero, precioNoche)`	`Libro(codigo, titulo)`
`reservar()` · `liberar()` · `estaOcupada()`	`prestar()` · `devolver()` · `estaPrestado()`
`precioEstancia(noches)`	`multa(dias)`
`Suite` (10 % de descuento desde 7 noches)	`Novedad` (multa más alta y recargo desde 7 días)
`Hotel`: `anadir`, `buscar`, `reservar`, `libres`	`Biblioteca`: `anadir`, `buscar`, `prestar`, `disponibles`
`guardarHotel` · `cargarHotel` · `hotel.txt`	`guardarBiblioteca` · `cargarBiblioteca` · `biblioteca.txt`

> **Atención**
> **No todo es igual.** Antes de copiar un test, lee el **comentario del** `.hpp` del método en `include/`: dice exactamente qué promete. **Lo que dice ese comentario es lo que manda**: si un test y el comentario no coinciden, el que está mal es el test.

3.4 El marcador: por dónde deberíais ir

Al terminar cada parte, comprobad que estáis aquí. En la columna de **Actions**, cada ✘ es un test que ha encontrado un error y el ✔ siguiente es su arreglo:

Al terminar…	Tests en vuestro ordenador	Errores encontrados y arreglados	En *Actions*
Parte 3	1 ✔	0 de 4	✔
Parte 4 (Libro)	10 ✔	1 de 4	✔ ✘ ✔
Parte 5 (Novedad)	15 ✔	2 de 4	✔ ✘ ✔ ✘ ✔
Parte 6 (Biblioteca)	21 ✔	3 de 4	✔ ✘ ✔ ✘ ✔ ✘ ✔
Parte 7 (guardar y cargar)	24 ✔	**4 de 4**	✔ ✘ ✔ ✘ ✔ ✘ ✔ ✘ ✔
Parte 8 (opcional)	25 ✔	4 de 4	… ✔

> **Truco**
> Si tenéis algún test de más o de menos, no pasa nada: lo importante es la columna de **errores**.
> **¿Atascados más de 10 minutos con un rojo?** Al final de la guía están las **Pistas**: para cada error, qué está mal, por qué y cómo se arregla. Usadlas **solo después** de haberlo intentado.

4. Pruebas de Libro: unitarias de caja negra (20 min)

Caja negra: el test se escribe con lo que **promete** el `.hpp`, sin mirar el `.cpp`. Trabajaréis con las dos ventanas de VS Code a la vez: a la izquierda la **referencia** (`tests/test_habitacion.cpp`), a la derecha la **biblioteca** (`tests/test_libro.cpp`).

4.1 Lee lo que promete Libro

En la biblioteca, abre `include/libro.hpp`. Estos son los comentarios que tienes que leer; los tests comprueban exactamente esto:

`include/libro.hpp (resumido)`

```
    // Lanza std::invalid_argument si el código no es positivo o el título está vacío.
    Libro(int codigo, const std::string& titulo);
    bool prestar();                              // false si ya estaba prestado
    void devolver();
    ...
    // Multa por devolver el libro tarde: 0,50 EUR por cada día de retraso.
    // 0 días de retraso -> 0 EUR. Lanza std::invalid_argument si dias < 0.
    virtual double multa(int dias) const;
```

4.2 Un test resuelto, como ejemplo

Abre `tests/test_libro.cpp`: ya tiene un test (`ConstructorGuardaLosDatos`). Vas a escribir los demás **debajo de la línea `// >>> AQUÍ <<<`**, uno detrás de otro. Así se copia y adapta uno: en la referencia, selecciona el test entero (desde `TEST(` hasta su `}`), `Ctrl+C` / `⌘C`, pégalo en la biblioteca y cambia lo marcado:

```
// En la REFERENCIA (tests/test_habitacion.cpp):
TEST(HabitacionTest, ReservarCambiaElEstado) {
    Habitacion h(101, 80.0);
    EXPECT_TRUE(h.reservar());
    EXPECT_TRUE(h.estaOcupada());
}

// En la BIBLIOTECA (tests/test_libro.cpp), debajo de >>> AQUÍ <<<:
TEST(LibroTest, PrestarCambiaElEstado) {
    Libro l(1, "El Quijote");
    EXPECT_TRUE(l.prestar());
    EXPECT_TRUE(l.estaPrestado());
}
```

Lo que hay que cambiar **siempre**: `HabitacionTest` → `LibroTest`, el nombre del test, el objeto (`Habitacion h(101, 80.0)` → `Libro l(1, "El Quijote")`) y los métodos (tabla del paso 3.3).

4.3 Los demás tests de Libro

Escríbelos en este orden. La columna de la derecha es la **línea clave** del test (la que comprueba); el resto se copia de la referencia y se adapta como en el ejemplo:

Test del hotel	Test de la biblioteca	Línea clave
`PrecioCeroLanzaExcepcion`	`CodigoCeroLanzaExcepcion`	`EXPECT_THROW(Libro(0, "El Quijote"), std::invalid_argument);`
`PrecioMinimoPositivoNoLanza`	`CodigoUnoNoLanza`	`EXPECT_NO_THROW(Libro(1, "El Quijote"));`
`NumeroNegativoLanzaExcepcion`	`TituloVacioLanzaExcepcion`	`EXPECT_THROW(Libro(1, ""), std::invalid_argument);`
`ReservarCambiaElEstado`	`PrestarCambiaElEstado`	El del ejemplo (4.2)
`ReservarDosVecesDevuelveFalse`	`PrestarDosVecesDevuelveFalse`	`l.prestar();` y después `EXPECT_FALSE(l.prestar());`
`LiberarDejaLaHabitacionLibre`	`DevolverLoDejaDisponible`	`l.prestar(); l.devolver();` y `EXPECT_FALSE(l.estaPrestado());`
`PrecioEstanciaMultiplicaPorLasNoches`	`MultaMultiplicaPorLosDias`	`EXPECT_DOUBLE_EQ(l.multa(4), 2.0);` (4 × 0,50)
`PrecioEstanciaConCeroNochesLanzaExcepcion`	**Dos tests**, porque aquí 0 días **sí** vale: `MultaConCeroDiasEsCero` `MultaConDiasNegativosLanzaExcepcion`	`EXPECT_DOUBLE_EQ(l.multa(0), 0.0);` `EXPECT_THROW(l.multa(-1), std::invalid_argument);`

> **Atención**
> **No copies un test de la referencia sin adaptarlo del todo.** Si queda un `Habitacion`, un `reservar` o un `HabitacionTest`, no compila. Y cada test necesita un **nombre distinto**.

4.4 Compila y ejecuta los tests

Cada dos o tres tests, **guarda** (`Ctrl+S` / `⌘S`), compila y ejecuta. Mientras todo va bien verás algo así:

Windows · PowerShell

```
PS C:\Users\alumno\PP\lab3-biblioteca> cmake --build build
…
PS C:\Users\alumno\PP\lab3-biblioteca> .\build\tests_gestor.exe
[==========] Running 4 tests from …
…
[==========] 4 tests from … ran. (1 ms total)
[  PASSED  ] 4 tests.
```

Mac · Terminal

```
alumno@MacBook-de-alumno lab3-biblioteca % cmake --build build
…
alumno@MacBook-de-alumno lab3-biblioteca % ./build/tests_gestor
[==========] Running 4 tests from …
…
[==========] 4 tests from … ran. (1 ms total)
[  PASSED  ] 4 tests.
```

Cuando tengas los 10 tests escritos, **uno de ellos sale en rojo**. Verás esto (el número de línea, `:41` , puede ser otro en tu fichero):

Windows · PowerShell

```
PS C:\Users\alumno\PP\lab3-biblioteca> cmake --build build
…
PS C:\Users\alumno\PP\lab3-biblioteca> .\build\tests_gestor.exe
[==========] Running 10 tests from …
…
C:/Users/alumno/PP/lab3-biblioteca/tests/test_libro.cpp:41: Failure
Value of: l.prestar()
  Actual: true
Expected: false
[  FAILED  ] LibroTest.PrestarDosVecesDevuelveFalse (0 ms)
…
[  PASSED  ] 9 tests.
[  FAILED  ] 1 test, listed below:
[  FAILED  ] LibroTest.PrestarDosVecesDevuelveFalse

 1 FAILED TEST
```

Mac · Terminal

```
alumno@MacBook-de-alumno lab3-biblioteca % cmake --build build
…
alumno@MacBook-de-alumno lab3-biblioteca % ./build/tests_gestor
[==========] Running 10 tests from …
…
/Users/alumno/PP/lab3-biblioteca/tests/test_libro.cpp:41: Failure
Value of: l.prestar()
  Actual: true
Expected: false
[  FAILED  ] LibroTest.PrestarDosVecesDevuelveFalse (0 ms)
…
[  PASSED  ] 9 tests.
[  FAILED  ] 1 test, listed below:
[  FAILED  ] LibroTest.PrestarDosVecesDevuelveFalse

 1 FAILED TEST
```

> **¿Qué ha pasado?**
>
> Se lee como en el paso 2.2: en la línea 41, `l.prestar()` ha devuelto `true` (*Actual*) y el test esperaba `false` (*Expected*). Es el **segundo** `prestar()` : el libro ya estaba prestado.
>
> **¿Está mal el test o está mal la clase?** El `.hpp` dice `// false si ya estaba prestado` . El test hace justo lo que dice el comentario, así que **el test está bien y la clase está mal**: has encontrado el **primer error**.

> **✘ Si en vez de eso ves…**
>
> **Error de compilación** (`'Habitacion' was not declared, has no member named 'reservar'` …): te has dejado algo del hotel sin cambiar; ve a la línea que dice el error.
>
> **Otro test en rojo** (no `PrestarDosVecesDevuelveFalse`): compara ese test con la línea clave de la tabla 4.3. Lo más probable es que el test esté mal copiado: corrígelo.

4.5 Del rojo al verde

Paso A · Sube el test en rojo (todavía **no** arregles nada):

Windows y Mac (igual en los dos)

```
PS C:\Users\alumno\PP\lab3-biblioteca> git add tests/test_libro.cpp
PS C:\Users\alumno\PP\lab3-biblioteca> git commit -m "test(libro): pruebas de caja negra de Libro"
…
PS C:\Users\alumno\PP\lab3-biblioteca> git push
…
```

Paso B · Míralo en GitHub: pestaña **Actions** → la ejecución nueva termina con una ✘ **roja**. Pulsa en ella → **build-and-test** → despliega el paso **Tests**: es el mismo rojo que en tu ordenador.

Paso C · Busca el error. Abre `src/libro.cpp`, busca la función `Libro::prestar` y léela línea a línea con el comentario del `.hpp` al lado. ¿Hace lo que dice el comentario?

Paso D · Arréglalo en `src/libro.cpp`, guarda, compila y ejecuta los tests hasta que estén **todos en verde**. Si llevas 10 minutos sin encontrarlo: **Pistas · Error 1**, al final de la guía.

Paso E · Sube el arreglo, con un mensaje que diga qué has arreglado:

Windows y Mac (igual en los dos)

```
PS C:\Users\alumno\PP\lab3-biblioteca> git add src/libro.cpp
PS C:\Users\alumno\PP\lab3-biblioteca> git commit -m "fix(libro): <qué has arreglado>"
…
PS C:\Users\alumno\PP\lab3-biblioteca> git push
…
```

En **Actions**, la ejecución nueva termina en ✔ **verde**.

✔ **Deberías ver**

`[ PASSED ] 10 tests.` en tu ordenador y, en **Actions**, una ✘ roja (el test) seguida de un ✔ verde (el arreglo). **Primer error encontrado. Quedan 3.**

✘ **Si en vez de eso ves…**

`git push` pide usuario y contraseña, o da `Permission denied / 403`: el repositorio es de tu pareja y no te ha añadido como colaborador (paso 3.1, punto 4), o tienes otra cuenta de GitHub abierta. Trabajad en el ordenador de quien creó el repositorio.

`Author identity unknown`: configura Git como en la práctica 1: `git config --global user.name "Tu Nombre"` y `git config --global user.email "tu@email"`, y repite el `git commit`.

5. Pruebas de Novedad: caja blanca y un fichero de tests nuevo (15 min)

Ahora el fichero de tests **no existe**: hay que crearlo y **decirle a CMake que lo compile**. Es lo mismo que tendréis que hacer en vuestro proyecto con cada clase nueva.

5.1 Crear el fichero y añadirlo al CMakeLists.txt

1. En la ventana de la biblioteca, botón derecho sobre la carpeta `tests` → **Nuevo archivo** → escribe `test_novedad.cpp` → `Enter`.
2. Copia dentro estas líneas (son las de `test_suite.cpp` de la referencia, con `suite.hpp` cambiado por `novedad.hpp`):

`tests/test_novedad.cpp (las primeras líneas: cópialas tal cual)`

```
// tests/test_novedad.cpp — PRUEBAS UNITARIAS de Novedad (CAJA BLANCA).
#include <gtest/gtest.h>
#include <memory>
#include "novedad.hpp"
```

3. Abre el `CMakeLists.txt` (en la raíz de la biblioteca), baja hasta `add_executable(tests_gestor` y añade la línea marcada. **Guarda.**

`CMakeLists.txt`

```
# >>> AQUÍ <<<  Cada vez que creéis un fichero de tests nuevo en tests/, añadidlo aquí.
add_executable(tests_gestor
    tests/test_libro.cpp
    tests/test_novedad.cpp
    ${FUENTES}
)
```

5.2 Qué hace el código de Novedad

Caja blanca significa escribir el test **mirando el código**. Abre `include/novedad.hpp` y `src/novedad.cpp`:

`include/novedad.hpp`

```
    // 1 EUR por cada día de retraso y, si el retraso es de 7 días o más (7 incluido),
    // un recargo fijo de 5 EUR. Lanza std::invalid_argument si dias < 0.
    double multa(int dias) const override;
```

En `src/novedad.cpp`, la multa tiene un `if` que decide si hay recargo. Hay que probar los **dos caminos** del `if` (sin recargo y con recargo) y, sobre todo, **el límite**: el 7. Es justo lo que hace `test_suite.cpp` con las 7 noches.

5.3 Un test resuelto, como ejemplo

```
// En la REFERENCIA (tests/test_suite.cpp):
TEST(SuiteTest, SinDescuentoConSeisNoches) {          // camino 1: el if NO se cumple
    Suite s(301, 200.0);
    EXPECT_DOUBLE_EQ(s.precioEstancia(6), 1200.0);
}

// En la BIBLIOTECA (tests/test_novedad.cpp), debajo de los #include:
TEST(NovedadTest, SinRecargoConSeisDias) {            // camino 1: el if NO se cumple
    Novedad n(3, "Dune");
    EXPECT_DOUBLE_EQ(n.multa(6), 6.0);                // 6 x 1 EUR, sin recargo
}
```

5.4 Los demás tests de Novedad

Test de la Suite	Test de la Novedad	Línea clave	Cuenta
`SinDescuentoConSeisNoches`	`SinRecargoConSeisDias`	El del ejemplo (5.3)	6 × 1 = 6
`ConDescuentoJustoConSieteNoches`	`ConRecargoJustoConSieteDias`	`EXPECT_DOUBLE_EQ(n.multa(7), 12.0);`	7 × 1 + 5 = 12
`ConDescuentoConDiezNoches`	`ConRecargoConDiezDias`	`EXPECT_DOUBLE_EQ(n.multa(10), 15.0);`	10 × 1 + 5 = 15
`CeroNochesLanzaExcepcion`	`DiasNegativosLanzaExcepcion`	`EXPECT_THROW(n.multa(-1), std::invalid_argument);`	−1 no vale
`PolimorfismoDesdeUnPunteroALaBase`	`PolimorfismoDesdeUnPunteroALaBase`	`std::unique_ptr<Libro> l = std::make_unique<Novedad>(3, "Dune");` `EXPECT_EQ(l->tipo(), "Novedad");` `EXPECT_DOUBLE_EQ(l->multa(6), 6.0);`	Se llama a la multa de **Novedad** (1 €/día), no a la de Libro (0,50)

5.5 Compila y ejecuta los tests

Guarda los dos ficheros (`test_novedad.cpp` y `CMakeLists.txt`), compila y ejecuta. Con los 5 tests escritos, uno sale en rojo:

Windows · PowerShell

```
PS C:\Users\alumno\PP\lab3-biblioteca> cmake --build build
…
PS C:\Users\alumno\PP\lab3-biblioteca> .\build\tests_gestor.exe
[==========] Running 15 tests from …
…
C:/Users/alumno/PP/lab3-biblioteca/tests/test_novedad.cpp:13: Failure
Expected equality of these values:
  n.multa(7)
    Which is: 7
  12.0
    Which is: 12
[  FAILED  ] NovedadTest.ConRecargoJustoConSieteDias (0 ms)
…
[  PASSED  ] 14 tests.
[  FAILED  ] 1 test, listed below:
[  FAILED  ] NovedadTest.ConRecargoJustoConSieteDias

 1 FAILED TEST
```

Mac · Terminal

```
alumno@MacBook-de-alumno lab3-biblioteca % cmake --build build
…
alumno@MacBook-de-alumno lab3-biblioteca % ./build/tests_gestor
[==========] Running 15 tests from …
…
/Users/alumno/PP/lab3-biblioteca/tests/test_novedad.cpp:13: Failure
Expected equality of these values:
  n.multa(7)
    Which is: 7
  12.0
    Which is: 12
[  FAILED  ] NovedadTest.ConRecargoJustoConSieteDias (0 ms)
…
[  PASSED  ] 14 tests.
[  FAILED  ] 1 test, listed below:
[  FAILED  ] NovedadTest.ConRecargoJustoConSieteDias

 1 FAILED TEST
```

¿Qué ha pasado?

Con 7 días, `multa` devuelve 7 y el test esperaba 12: **falta el recargo**. El `.hpp` dice «7 días o más (**7 incluido**)», así que el test está bien: **segundo error**. Fíjate en que con 6 y con 10 días el resultado es correcto: **solo el test del límite lo encuentra**. Por eso, en caja blanca, siempre se prueba el valor exacto del `if`.

✘ Si en vez de eso ves…

No aparecen los 15 tests (sigue diciendo `Running 10 tests`): no has guardado el `CMakeLists.txt` o la línea está fuera de `add_executable(tests_gestor …)`.

`fatal error: suite.hpp: No such file or directory`: cambia el `#include` por `"novedad.hpp"`.

`'Suite' was not declared`: queda algo de la Suite sin adaptar.

5.6 Del rojo al verde

Paso A · Sube el test en rojo (todavía **no** arregles nada):

Windows y Mac (igual en los dos)

```
PS C:\Users\alumno\PP\lab3-biblioteca> git add tests/test_novedad.cpp CMakeLists.txt
PS C:\Users\alumno\PP\lab3-biblioteca> git commit -m "test(novedad): pruebas de caja blanca de Novedad"
…
PS C:\Users\alumno\PP\lab3-biblioteca> git push
…
```

Paso B · Míralo en GitHub: pestaña **Actions** → la ejecución nueva termina con una **✘ roja**. Pulsa en ella → **build-and-test** → despliega el paso **Tests**: es el mismo rojo que en tu ordenador.

Paso C · Busca el error. Abre `src/novedad.cpp`, busca la función `Novedad::multa` y léela línea a línea con el comentario del `.hpp` al lado. ¿Hace lo que dice el comentario?

Paso D · Arréglalo en `src/novedad.cpp`, guarda, compila y ejecuta los tests hasta que estén **todos en verde**. Si llevas 10 minutos sin encontrarlo: **Pistas · Error 2**, al final de la guía.

Paso E · Sube el arreglo, con un mensaje que diga qué has arreglado:

Windows y Mac (igual en los dos)

```
PS C:\Users\alumno\PP\lab3-biblioteca> git add src/novedad.cpp
PS C:\Users\alumno\PP\lab3-biblioteca> git commit -m "fix(novedad): <qué has arreglado>"
…
PS C:\Users\alumno\PP\lab3-biblioteca> git push
…
```

En **Actions**, la ejecución nueva termina en **✔ verde**.

> ✔ **Deberías ver**
> `[ PASSED ] 15 tests.` y, en **Actions**, otra ✘ seguida de ✔. **Segundo error encontrado. Quedan 2.**

6. Pruebas de Biblioteca: un fixture con TEST_F (20 min)

Un ***fixture*** es una clase que **prepara los mismos datos** para varios tests. Cada `TEST_F` empieza con una biblioteca **nueva**, con tres libros, preparada en `SetUp()`: así no se repite en cada test, y lo que haga un test no afecta a los demás.

6.1 Crear el fichero, el fixture y el primer test

1. Botón derecho sobre `tests` → **Nuevo archivo** → `test_biblioteca.cpp`.
2. Copia dentro **todo esto** (es el principio de `test_hotel.cpp`, ya adaptado):

`tests/test_biblioteca.cpp (cópialo tal cual)`

```
// tests/test_biblioteca.cpp — PRUEBAS UNITARIAS de Biblioteca con un FIXTURE (TEST_F).
#include <gtest/gtest.h>
#include <memory>
#include <stdexcept>
#include "biblioteca.hpp"
#include "novedad.hpp"

class BibliotecaTest : public ::testing::Test {
protected:
    void SetUp() override {                       // se ejecuta ANTES de cada TEST_F
        biblioteca.anadir(std::make_unique<Libro>(1, "El Quijote"));
        biblioteca.anadir(std::make_unique<Libro>(2, "La Regenta"));
        biblioteca.anadir(std::make_unique<Novedad>(3, "Dune"));
    }
    Biblioteca biblioteca;
};

TEST_F(BibliotecaTest, TotalCuentaTodosLosLibros) {
    EXPECT_EQ(biblioteca.total(), 3u);            // 3u: total() devuelve un size_t (sin signo)
    EXPECT_EQ(biblioteca.disponibles(), 3);
}
```

3. Añade `tests/test_biblioteca.cpp` al `CMakeLists.txt`, debajo de `tests/test_novedad.cpp`, y guarda.

`CMakeLists.txt`

```
add_executable(tests_gestor
    tests/test_libro.cpp
    tests/test_novedad.cpp
    tests/test_biblioteca.cpp
    ${FUENTES}
)
```

Compila y ejecuta: deberías ver `[ PASSED ] 16 tests.` (los 15 de antes y este).

6.2 Un test resuelto, como ejemplo

En los `TEST_F` **no se crea** la biblioteca: ya existe (se llama `biblioteca`) y tiene los libros 1, 2 y 3.

```
// En la REFERENCIA (tests/test_hotel.cpp):
TEST_F(HotelTest, BuscarUnaQueExisteLaDevuelve) {
    Habitacion* h = hotel.buscar(301);
    ASSERT_NE(h, nullptr);                        // ASSERT: si falla, el test PARA aquí
    EXPECT_EQ(h->getNumero(), 301);
    EXPECT_EQ(h->tipo(), "Suite");
}

// En la BIBLIOTECA (tests/test_biblioteca.cpp):
TEST_F(BibliotecaTest, BuscarUnoQueExisteLoDevuelve) {
    Libro* l = biblioteca.buscar(3);
    ASSERT_NE(l, nullptr);                        // ASSERT: si falla, el test PARA aquí
    EXPECT_EQ(l->getTitulo(), "Dune");
    EXPECT_EQ(l->tipo(), "Novedad");
}
```

> **¿Qué ha pasado?**
> **¿Por qué `ASSERT` y no `EXPECT`?** Si `buscar` devolviera `nullptr` y el test siguiera, `l->getTitulo()` rompería el programa de tests entero. `ASSERT` para el test en ese punto.

6.3 Los demás tests de Biblioteca

Test del hotel	Test de la biblioteca	Líneas clave
`BuscarUnaQueNoExisteDevuelveNullptr`	`BuscarUnoQueNoExisteDevuelveNullptr`	`EXPECT_EQ(biblioteca.buscar(999), nullptr);`
`AnadirUnNumeroRepetidoLanzaExcepcion`	`AnadirUnCodigoRepetidoLanzaExcepcion`	`EXPECT_THROW(biblioteca.anadir(std::make_unique<Libro>(1, "Otro")), std::invalid_argument);` `EXPECT_EQ(biblioteca.total(), 3u);`
`ReservarReduceLasLibres`	`PrestarReduceLosDisponibles`	`EXPECT_TRUE(biblioteca.prestar(2));` `EXPECT_EQ(biblioteca.disponibles(), 2);` `EXPECT_TRUE(biblioteca.buscar(2)->estaPrestado());`
`ReservarUnaQueNoExisteLanzaExcepcion`	`PrestarUnoQueNoExisteLanzaExcepcion`	`EXPECT_THROW(biblioteca.prestar(999), std::out_of_range);`

> **Truco**
> El código 1 ya está en la biblioteca (lo añade `SetUp`). Por eso, añadir **otro** libro con el código 1 tiene que lanzar la excepción que promete el `.hpp`, y el total tiene que seguir siendo 3.

6.4 Compila y ejecuta los tests

Con los 6 tests escritos, uno sale en rojo, esta vez con **dos** fallos dentro del mismo test:

Windows · PowerShell

```
PS C:\Users\alumno\PP\lab3-biblioteca> cmake --build build
…
PS C:\Users\alumno\PP\lab3-biblioteca> .\build\tests_gestor.exe
[==========] Running 21 tests from …
…
C:/Users/alumno/PP/lab3-biblioteca/tests/test_biblioteca.cpp:35: Failure
Expected: biblioteca.anadir(std::make_unique<Libro>(1, "Otro"))
  throws an exception of type std::invalid_argument.
  Actual: it throws nothing.
C:/Users/alumno/PP/lab3-biblioteca/tests/test_biblioteca.cpp:36: Failure
Expected equality of these values:
  biblioteca.total()
    Which is: 4
  3u
    Which is: 3
[  FAILED  ] BibliotecaTest.AnadirUnCodigoRepetidoLanzaExcepcion (0 ms)
…
[  PASSED  ] 20 tests.
[  FAILED  ] 1 test, listed below:
[  FAILED  ] BibliotecaTest.AnadirUnCodigoRepetidoLanzaExcepcion

 1 FAILED TEST
```

Mac · Terminal

```
alumno@MacBook-de-alumno lab3-biblioteca % cmake --build build
…
alumno@MacBook-de-alumno lab3-biblioteca % ./build/tests_gestor
[==========] Running 21 tests from …
…
/Users/alumno/PP/lab3-biblioteca/tests/test_biblioteca.cpp:35: Failure
Expected: biblioteca.anadir(std::make_unique<Libro>(1, "Otro"))
  throws an exception of type std::invalid_argument.
  Actual: it throws nothing.
/Users/alumno/PP/lab3-biblioteca/tests/test_biblioteca.cpp:36: Failure
Expected equality of these values:
  biblioteca.total()
    Which is: 4
  3u
    Which is: 3
[  FAILED  ] BibliotecaTest.AnadirUnCodigoRepetidoLanzaExcepcion (0 ms)
…
[  PASSED  ] 20 tests.
[  FAILED  ] 1 test, listed below:
[  FAILED  ] BibliotecaTest.AnadirUnCodigoRepetidoLanzaExcepcion

 1 FAILED TEST
```

> **¿Qué ha pasado?**
> Línea 35: `anadir` con un código repetido **no lanza nada** (*it throws nothing*). Línea 36: y además lo ha añadido: hay 4 libros, dos con el código 1. El `.hpp` de `Biblioteca::anadir` dice `// Lanza std::invalid_argument si ya hay un libro con ese código.` El test está bien: **tercer error**.

6.5 Del rojo al verde

Paso A · Sube el test en rojo (todavía **no** arregles nada):

Windows y Mac (igual en los dos)

```
PS C:\Users\alumno\PP\lab3-biblioteca> git add tests/test_biblioteca.cpp CMakeLists.txt
PS C:\Users\alumno\PP\lab3-biblioteca> git commit -m "test(biblioteca): pruebas de Biblioteca con fixture"
…
PS C:\Users\alumno\PP\lab3-biblioteca> git push
…
```

Paso B · Míralo en GitHub: pestaña **Actions** → la ejecución nueva termina con una ✘ **roja**. Pulsa en ella → **build-and-test** → despliega el paso **Tests**: es el mismo rojo que en tu ordenador.

Paso C · Busca el error. Abre `src/biblioteca.cpp`, busca la función `Biblioteca::anadir` y léela línea a línea con el comentario del `.hpp` al lado. ¿Hace lo que dice el comentario?

Paso D · Arréglalo en `src/biblioteca.cpp`, guarda, compila y ejecuta los tests hasta que estén **todos en verde**. Si llevas 10 minutos sin encontrarlo: **Pistas · Error 3**, al final de la guía.

Paso E · Sube el arreglo, con un mensaje que diga qué has arreglado:

Windows y Mac (igual en los dos)

```
PS C:\Users\alumno\PP\lab3-biblioteca> git add src/biblioteca.cpp
PS C:\Users\alumno\PP\lab3-biblioteca> git commit -m "fix(biblioteca): <qué has arreglado>"
…
PS C:\Users\alumno\PP\lab3-biblioteca> git push
…
```

En **Actions**, la ejecución nueva termina en ✔ **verde**.

> ✔ **Deberías ver**
> `[ PASSED ] 21 tests.` y otra ✘ seguida de ✔ en **Actions**. **Tercer error encontrado. Queda 1.**

7. Prueba de integración: guardar y cargar (15 min)

Una prueba de **integración** no prueba una clase sola, sino que **varias piezas funcionan juntas**: `Biblioteca`, `Libro`, `Novedad`, `guardarBiblioteca`, `cargarBiblioteca` y un fichero de verdad en el disco.

7.1 Crear el fichero con el test de guardar y cargar

1. Botón derecho sobre `tests` → **Nuevo archivo** → `test_almacen.cpp`.
2. Copia dentro **todo esto** (es `test_almacen.cpp` de la referencia, con el primer test ya adaptado):

tests/test_almacen.cpp (cópialo tal cual)

```
// tests/test_almacen.cpp — PRUEBAS DE INTEGRACIÓN: Biblioteca + Libro + Novedad + fichero.
#include <gtest/gtest.h>
#include <cstdio>
#include <fstream>
#include <memory>
#include "almacen.hpp"
#include "novedad.hpp"

TEST(AlmacenTest, GuardarYCargarDevuelveLoMismo) {
    // PREPARAR: una biblioteca con un Libro y una Novedad prestada, guardada en un fichero
    Biblioteca original;
    original.anadir(std::make_unique<Libro>(1, "El Quijote"));
    original.anadir(std::make_unique<Novedad>(3, "Dune"));
    original.prestar(3);
    guardarBiblioteca(original, "test_biblioteca.txt");

    // ACTUAR: cargarla desde el fichero
    Biblioteca cargada = cargarBiblioteca("test_biblioteca.txt");

    // COMPROBAR: ha vuelto TODO igual
    ASSERT_EQ(cargada.total(), 2u);
    Libro* novedad = cargada.buscar(3);
    ASSERT_NE(novedad, nullptr);
    EXPECT_EQ(novedad->tipo(), "Novedad");        // se ha vuelto a crear como Novedad, no como Libro
    EXPECT_EQ(novedad->getTitulo(), "Dune");
    EXPECT_TRUE(novedad->estaPrestado());
    EXPECT_FALSE(cargada.buscar(1)->estaPrestado());
    std::remove("test_biblioteca.txt");           // borrar el fichero de prueba
}
```

3. Añade `tests/test_almacen.cpp` al `CMakeLists.txt`, debajo de `tests/test_biblioteca.cpp`, y guarda.

7.2 Los otros dos tests

Cópialos de `test_almacen.cpp` de la referencia y cambia lo marcado:

tests/test_almacen.cpp (debajo del primero)

```
TEST(AlmacenTest, FicheroQueNoExisteLanzaExcepcion) {
    EXPECT_THROW(cargarBiblioteca("no_existe.txt"), FicheroException);
}

TEST(AlmacenTest, LineaMalEscritaLanzaExcepcion) {
    {
        std::ofstream f("test_roto.txt");
        f << "Libro;1;El Quijote;0\nLibro;abc;La Regenta;0\n";   // "abc" no es un código
    }   // aquí se cierra el fichero: ya se puede leer
    EXPECT_THROW(cargarBiblioteca("test_roto.txt"), FicheroException);
    std::remove("test_roto.txt");
}
```

¿Qué ha pasado?

El segundo test **escribe a propósito un fichero roto** (una línea con `abc` donde va el código) y comprueba que `cargarBiblioteca` protesta con una `FicheroException`. Las llaves `{ … }` sirven para que el fichero se cierre antes de leerlo. Los ficheros de prueba se crean en la carpeta desde la que ejecutas los tests; por eso cada test borra lo que crea.

7.3 Compila y ejecuta los tests

Windows · PowerShell

```
PS C:\Users\alumno\PP\lab3-biblioteca> cmake --build build
…
PS C:\Users\alumno\PP\lab3-biblioteca> .\build\tests_gestor.exe
[==========] Running 24 tests from …
…
C:/Users/alumno/PP/lab3-biblioteca/tests/test_almacen.cpp:24: Failure
Expected equality of these values:
  novedad->tipo()
    Which is: "Libro"
  "Novedad"
[  FAILED  ] AlmacenTest.GuardarYCargarDevuelveLoMismo (0 ms)
…
[  PASSED  ] 23 tests.
[  FAILED  ] 1 test, listed below:
[  FAILED  ] AlmacenTest.GuardarYCargarDevuelveLoMismo

 1 FAILED TEST
```

Mac · Terminal

```
alumno@MacBook-de-alumno lab3-biblioteca % cmake --build build
…
alumno@MacBook-de-alumno lab3-biblioteca % ./build/tests_gestor
[==========] Running 24 tests from …
…
/Users/alumno/PP/lab3-biblioteca/tests/test_almacen.cpp:24: Failure
Expected equality of these values:
  novedad->tipo()
    Which is: "Libro"
  "Novedad"
[  FAILED  ] AlmacenTest.GuardarYCargarDevuelveLoMismo (0 ms)
…
[  PASSED  ] 23 tests.
[  FAILED  ] 1 test, listed below:
[  FAILED  ] AlmacenTest.GuardarYCargarDevuelveLoMismo

 1 FAILED TEST
```

¿Qué ha pasado?

Se guardó una `Novedad` y, al cargarla, ha vuelto como `Libro`. El libro está, con su título y prestado, pero ha **perdido su tipo** (y con él, su multa). El formato del fichero está en `include/almacen.hpp`: `Novedad;3;Dune;1`. **Cuarto error**.
Cada pieza, probada sola, funciona; **juntas, no**. Eso es lo que encuentran las pruebas de integración y no ven las unitarias.

7.4 Del rojo al verde

Paso A · Sube el test en rojo (todavía **no** arregles nada):

Windows y Mac (igual en los dos)

```
PS C:\Users\alumno\PP\lab3-biblioteca> git add tests/test_almacen.cpp CMakeLists.txt
PS C:\Users\alumno\PP\lab3-biblioteca> git commit -m "test(almacen): pruebas de integracion de guardar y cargar"
…
PS C:\Users\alumno\PP\lab3-biblioteca> git push
…
```

Paso B · Míralo en GitHub: pestaña **Actions** → la ejecución nueva termina con una **✘ roja**. Pulsa en ella → **build-and-test** → despliega el paso **Tests**: es el mismo rojo que en tu ordenador.

Paso C · Busca el error. Abre `src/almacen.cpp`, busca la función `cargarBiblioteca` y léela línea a línea con el comentario del `.hpp` al lado. ¿Hace lo que dice el comentario?

Paso D · Arréglalo en `src/almacen.cpp`, guarda, compila y ejecuta los tests hasta que estén **todos en verde**. Si llevas 10 minutos sin encontrarlo: **Pistas · Error 4**, al final de la guía.

Paso E · Sube el arreglo, con un mensaje que diga qué has arreglado:

Windows y Mac (igual en los dos)

```
PS C:\Users\alumno\PP\lab3-biblioteca> git add src/almacen.cpp
PS C:\Users\alumno\PP\lab3-biblioteca> git commit -m "fix(almacen): <qué has arreglado>"
…
PS C:\Users\alumno\PP\lab3-biblioteca> git push
…
```

En **Actions**, la ejecución nueva termina en ✔ **verde**.

> ✔ **Deberías ver**
>
> **Los 4 errores encontrados y arreglados:** `[ PASSED ] 24 tests.` en tu ordenador y ✔ verde en **Actions**. En *Actions* se ve la historia de la práctica: cuatro rojos, cada uno seguido de su arreglo.

8. (Opcional) Prueba de rendimiento

No comprueba *qué* devuelve el programa, sino **cuánto tarda** con muchos datos. Crea `tests/test_rendimiento.cpp`, copia dentro todo el de la referencia, cambia lo marcado y añádelo al `CMakeLists.txt`:

tests/test_rendimiento.cpp

```
#include <gtest/gtest.h>
#include <chrono>
#include <memory>
#include "biblioteca.hpp"

TEST(RendimientoTest, BuscarEnUnaBibliotecaGrandeTardaMenosDeUnSegundo) {
    Biblioteca biblioteca;
    for (int n = 1; n <= 2000; n++) {
        biblioteca.anadir(std::make_unique<Libro>(n, "Libro"));
    }

    auto inicio = std::chrono::steady_clock::now();
    int encontrados = 0;
    for (int n = 1; n <= 1000; n++) {
        if (biblioteca.buscar(n * 2) != nullptr) encontrados++;
    }
    auto fin = std::chrono::steady_clock::now();
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(fin - inicio).count();

    EXPECT_EQ(encontrados, 1000);
    EXPECT_LT(ms, 1000) << "La busqueda ha tardado " << ms << " ms";
}
```

Deberías ver `[ PASSED ] 25 tests.` Súbelo con `git add tests/test_rendimiento.cpp CMakeLists.txt`, `git commit -m "test(rendimiento): busqueda en una biblioteca grande"` y `git push`.

> **¿Qué ha pasado?**
>
> El límite (`EXPECT_LT(ms, 1000)`, menos de un segundo) es muy generoso a propósito: el CI se ejecuta en máquinas compartidas, y la prueba sirve para detectar que algo se ha vuelto **mucho** más lento, no para medir con precisión.

Pistas · Los 4 errores, explicados

> **Atención**
>
> **Usad esta página solo si lleváis más de 10 minutos atascados con un rojo.** Antes: leed el rojo (qué se obtiene y qué se esperaba), leed el comentario del `.hpp` y leed la función línea a línea. Encontrar el error vosotros es lo que se aprende en esta práctica.

Error 1 · lo encuentra `LibroTest.PrestarDosVecesDevuelveFalse`

Qué está mal. `Libro::prestar()` pone el libro como prestado y devuelve `true` **siempre**, sin mirar antes si ya estaba prestado.

Por qué. El `.hpp` promete `// false si ya estaba prestado`: prestar un libro que ya está prestado tiene que fallar (devolver `false`). Sin esa comprobación, la biblioteca prestaría el mismo libro a dos personas.

Corrección, en `src/libro.cpp`:

Antes (mal)

```
bool Libro::prestar() {
    prestado_ = true;
    return true;
}
```

Después (bien)

```
bool Libro::prestar() {
    if (prestado_) return false;
    prestado_ = true;
    return true;
}
```

Mensaje del commit del arreglo: `fix(libro): prestar devuelve false si ya estaba prestado`

Error 2 · lo encuentra `NovedadTest.ConRecargoJustoConSieteDias`

Qué está mal. El `if` del recargo usa `dias > 7` (estrictamente mayor), así que con 7 días **no** se aplica el recargo.

Por qué. El `.hpp` dice «7 días o más (**7 incluido**)»: con 7 días tiene que haber recargo. `>` deja fuera el 7; `>=` lo incluye. Es el error de **valor límite** más típico, y por eso solo lo encuentra el test del límite: con 6 o con 10 días, `>` y `>=` dan lo mismo.

Corrección, en `src/novedad.cpp`:

Antes (mal)

```
    if (dias > 7) {
        importe = importe + 5.0;                // recargo
    }
```

Después (bien)

```
    if (dias >= 7) {
        importe = importe + 5.0;                // recargo
    }
```

Mensaje del commit del arreglo: `fix(novedad): el recargo se aplica tambien con 7 dias`

Error 3 · lo encuentra `BibliotecaTest.AnadirUnCodigoRepetidoLanzaExcepcion`

Qué está mal. `Biblioteca::anadir` mete el libro en el vector **sin comprobar** si ya hay otro con el mismo código.

Por qué. El `.hpp` promete `// Lanza std::invalid_argument si ya hay un libro con ese código.` Con dos libros con el código 1, `buscar(1)` encontraría siempre el primero y el segundo no se podría prestar nunca. La comprobación se hace con `buscar`, que ya existe, y tiene que ir **antes** del `push_back`: después del `std::move`, `libro` ya está vacío.

Corrección, en `src/biblioteca.cpp`:

Antes (mal)

```
void Biblioteca::anadir(std::unique_ptr<Libro> libro) {
    libros_.push_back(std::move(libro));
}
```

Después (bien)

```
void Biblioteca::anadir(std::unique_ptr<Libro> libro) {
    if (buscar(libro->getCodigo()) != nullptr) {
        throw std::invalid_argument("Ya existe el libro "
            + std::to_string(libro->getCodigo()));
    }
    libros_.push_back(std::move(libro));
}
```

Mensaje del commit del arreglo: `fix(biblioteca): anadir lanza excepcion si el codigo esta repetido`

Error 4 · lo encuentra `AlmacenTest.GuardarYCargarDevuelveLoMismo`

Qué está mal. `cargarBiblioteca` compara el tipo leído del fichero con `"novedad"`, en **minúsculas**.

Por qué. `guardarBiblioteca` escribe lo que devuelve `tipo()`, que es `"Novedad"`, con **N mayúscula** (`Novedad;3;Dune;1`). La comparación de `string` distingue mayúsculas y minúsculas: `"Novedad" == "novedad"` es `false`, así que se va por el `else` y la novedad se crea como `Libro`. Guardar funciona y cargar funciona, pero **no se ponen de acuerdo**: un error de integración.

Corrección, en `src/almacen.cpp`:

Antes (mal)

```
            if (tipo == "novedad") {
```

Después (bien)

```
            if (tipo == "Novedad") {
```

Mensaje del commit del arreglo: `fix(almacen): cargar reconoce las novedades`

Para casa · Lo mismo, en el proyecto de vuestro grupo

> **Atención**
>
> **Viernes 20 de noviembre, 23:59:** se congela el repositorio de la Entrega 2. Tiene que tener **al menos 5 tests** que pasen y el CI de **Actions en verde**. En la defensa (lunes 23) os puedo preguntar por cualquier test: qué comprueba y por qué fallaría si la clase estuviera mal.

1. Por cada clase de vuestro proyecto, un fichero `tests/test_<clase>.cpp`, añadido a `add_executable(tests_gestor …)`.
2. Copiad los tests de la referencia que más se parezcan a vuestra clase y adaptadlos, como hoy. La tabla de abajo dice cuáles, como mínimo.
3. Antes de cada `git push`: `cmake --build build` y `ctest --output-on-failure` en vuestro ordenador.
4. Cuando hayáis terminado con la clase de ejemplo de la plantilla, borradla junto con su test (y sus líneas del `CMakeLists.txt`).

Prueba mínima recomendada	Modelo en la referencia
El constructor guarda bien los datos	`HabitacionTest.ConstructorGuardaLosDatos`
Un dato no válido lanza una excepción (valor límite)	`HabitacionTest.PrecioCeroLanzaExcepcion`
Un método cambia el estado del objeto	`HabitacionTest.ReservarCambiaElEstado`
Si hay un `if` con un límite: los dos caminos y el límite	`SuiteTest.*`
Si hay herencia: el polimorfismo desde un puntero a la base	`SuiteTest.PolimorfismoDesdeUnPunteroALaBase`
La persistencia (obligatoria): guardar y cargar devuelve lo mismo	`AlmacenTest.GuardarYCargarDevuelveLoMismo`

> **Con Claude**
>
> Pegad una de vuestras clases (el `.hpp` y el `.cpp`) y uno de vuestros tests, y pedidle: *«Propón 3 tests de GoogleTest más para esta clase, en el mismo estilo que este. Dime para cada uno si es de caja negra o de caja blanca y qué valor límite prueba»*. **Antes de añadir un test, tenéis que saber explicar qué comprueba y por qué fallaría si la clase estuviera mal.**

Resumen: la chuleta de la práctica

Quiero...	Windows (PowerShell)	Mac (Terminal)
Configurar (1.ª vez)	`cmake -S . -B build -G Ninja`	`cmake -S . -B build`
Compilar (cada vez)	`cmake --build build`	`cmake --build build`
Ver cada test	`.\build\tests_gestor.exe`	`./build/tests_gestor`
Pasar los tests como el CI	`cd build`, `ctest --output-on-failure`, `cd ..`	igual
Pasar solo unos tests	`.\build\tests_gestor.exe --gtest_filter=Libro*`	`./build/tests_gestor --gtest_filter='Libro*'`
Subir el trabajo	`git add …`, `git commit -m "test(libro): …"`, `git push`	igual
Traer lo que ha subido tu pareja	`git pull`	igual
Empezar de cero (borrar build)	`Remove-Item -Recurse -Force build`	`rm -rf build`

Tipo de prueba	Qué es	En la práctica
Unitaria	Una clase (o una función) sola	`test_libro`, `test_novedad`, `test_biblioteca`
Caja negra	Se escribe con lo que **promete** el `.hpp`, sin mirar el código. Casos normales y valores límite	`test_libro`
Caja blanca	Se escribe **mirando el código**: un test por camino del `if`, y el límite	`test_novedad`
Integración	Varias piezas **juntas**	`test_almacen`
Rendimiento	**Cuánto tarda** con muchos datos	`test_rendimiento`

Si algo falla

Ves esto	Qué pasa	Qué hacer
`error: 'Habitacion' was not declared / has no member named …`	Queda algo del hotel sin adaptar	Ve a esa línea del test
`fatal error: suite.hpp: No such file or directory`	Falta cambiar el `#include`	`#include "novedad.hpp"`
Los tests nuevos no aparecen	El fichero no está en el `CMakeLists.txt`, o no lo has guardado	Paso 5, punto 4
`redefinition of 'class LibroTest_…_Test'`	Dos tests con el mismo grupo y nombre (al copiar, se te olvidó cambiarlo)	Cambia el nombre de uno
El programa de tests se cierra de golpe (*Segmentation fault*, código de salida raro)	Has usado un puntero `nullptr`	`ASSERT_NE(p, nullptr);` antes de `p->`
He cambiado el código y no cambia nada	No has guardado	● en la pestaña = sin guardar → `Ctrl+S`

Anexo · Instalar las herramientas (repaso de la Práctica 0)

Solo si en el paso 1.1 te falta alguna herramienta. Necesitas **conexión a internet** y unos **15-20 minutos**. Haz solo la parte de tu sistema y, dentro de ella, solo los pasos de lo que te falte. Al terminar, **cierra todas las terminales, abre una nueva** y repite las comprobaciones del paso 1.1.

Herramienta	Windows	Mac
Compilador `g++`	MSYS2 → **W1-W3**	Xcode Command Line Tools → **M1**
`cmake`	MSYS2 → **W2-W3**	cmake.org → **M2**
`ninja`	MSYS2 → **W2-W3**	no hace falta
`git`	git-scm.com → **W4**	viene con las Command Line Tools → **M1**
VS Code	code.visualstudio.com → **W5**	code.visualstudio.com → **M3**

Windows

W1. Instalar MSYS2 (el "gestor de paquetes" desde el que se instalan g++, CMake y Ninja):

1. Entra en **https://www.msys2.org** y pulsa el enlace del instalador, `msys2-x86_64-XXXXXXXX.exe` (la fecha del nombre cambia).
2. Ábrelo desde *Descargas* y pulsa **Siguiente** en todo, **sin cambiar la carpeta**: tiene que ser `C:\msys64`.
3. Al final, deja marcada **Run MSYS2 now** y pulsa **Finalizar**. Se abre una terminal **distinta** de PowerShell: la de MSYS2.

W2. Instalar g++, CMake y Ninja. En la terminal **MSYS2 MSYS** (tecla Windows → escribe MSYS2 → "MSYS2 MSYS"), primero actualiza:

Windows · terminal MSYS2 MSYS (no PowerShell)

```
$ pacman -Syu
```

Si pregunta `Proceed with installation? [Y/n]`, escribe `Y` y Enter. Si la ventana se cierra sola, es normal: ábrela otra vez y repite `pacman -Syu` hasta que diga `there is nothing to do`. Después instala las tres herramientas de una vez (confirma con `Y`):

Windows · terminal MSYS2 MSYS (no PowerShell)

```
$ pacman -S --needed mingw-w64-x86_64-gcc mingw-w64-x86_64-cmake mingw-w64-x86_64-ninja
```

W3. Añadir la carpeta de MSYS2 al PATH (para que PowerShell encuentre `g++`, `cmake` y `ninja`):

1. Tecla Windows → escribe **variables de entorno** → **Editar las variables de entorno del sistema** → botón **Variables de entorno…**
2. En la mitad de arriba (*Variables de usuario*), selecciona **Path** → **Editar…** → **Nuevo** → escribe exactamente `C:\msys64\mingw64\bin`
3. **Aceptar** en las tres ventanas.
4. **Cierra todas las ventanas de PowerShell** (y VS Code, si estaba abierto): solo las terminales nuevas ven el cambio.

W4. Instalar Git: entra en **https://git-scm.com/downloads/win**, pulsa **Click here to download**, abre el instalador y pulsa **Next** en todo (las opciones por defecto valen).

W5. Instalar VS Code (si no lo tienes): **https://code.visualstudio.com** → **Download for Windows**. Durante la instalación, deja marcada la casilla **Añadir a la RUTA (PATH)**.

> **✔ Deberías ver**
>
> En una PowerShell **nueva**, `g++ --version`, `cmake --version`, `ninja --version` y `git --version` responden con un número de versión (como en el paso 1.1).

> **✘ Si en vez de eso ves…**
>
> Siguen sin reconocerse: comprueba en W3 que la ruta está escrita **exactamente** como `C:\msys64\mingw64\bin` (sin espacios al final) y que has abierto una terminal nueva. Si aun así falla, reinicia el ordenador.
> En W2, `error: target not found`: hay una errata en el nombre de algún paquete. Copia el comando tal cual.

Mac

M1. Instalar g++ y Git (Xcode Command Line Tools). En la Terminal:

Mac · Terminal

```
alumno@MacBook-de-alumno ~ % xcode-select --install
```

Se abre una ventana: pulsa **Instalar**, acepta la licencia y espera (entre 5 y 15 minutos). Si en lugar de eso ves `command line tools are already installed`, ya los tienes: pasa a M2.

M2. Instalar CMake:

1. Entra en **https://cmake.org/download/**. En *Binary distributions*, descarga el fichero de macOS que termina en `macos-universal.dmg` (vale para chips Intel y Apple M).
2. Abre el `.dmg` y arrastra el icono de **CMake** a la carpeta **Applications**.
3. Abre **CMake** desde Aplicaciones (si macOS pregunta si quieres abrirla, pulsa **Abrir**).
4. **Este es el paso que más se olvida:** en el menú de arriba, **Tools → How to Install For Command Line Use**. Copia el comando que aparece, que es este:

Mac · Terminal

```
alumno@MacBook-de-alumno ~ % sudo "/Applications/CMake.app/Contents/bin/cmake-gui" --install
Password:
```

Pégalo en la Terminal y pulsa Enter. Te pide la contraseña de tu Mac: escríbela (**no se ve nada mientras escribes**, es normal) y pulsa Enter. Cierra la Terminal, abre una nueva y comprueba con `cmake --version`.

✘ Si en vez de eso ves...

`Failed to create symlink` o `No such file or directory` con `/usr/local/bin`: esa carpeta no existe en tu Mac. Créala con `sudo mkdir -p /usr/local/bin` y repite el comando `sudo "/Applications/CMake.app/…" --install`.
Si usas **Homebrew**, también vale `brew install cmake` en lugar de los pasos 1-4.

M3. Instalar VS Code (si no lo tienes): **https://code.visualstudio.com** → **Download for Mac**, arrastra la aplicación a **Aplicaciones** y ábrela. Después, para poder usar `code .`: ⌘ Cmd + ⇧ Shift + P → **Shell Command: Install 'code' command in PATH**.

✔ Deberías ver

En una Terminal **nueva**, `g++ --version`, `cmake --version` y `git --version` responden con un número de versión (como en el paso 1.1).